



Project 3

Calculating the Development of a Turbulent Boundary Layer

ME 5311

John Gomes

Professor Xinyu Zhao

May 2026

Introduction

The goal of this project is to obtain a numerical solution to the zero-pressure gradient turbulent boundary layer equations. The Crank-Nicolson scheme, along with a central difference method will be used to numerically solve following equations for continuity and boundary layer momentum.

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0 \quad (1)$$

$$u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} = \frac{\partial}{\partial y} \left((\nu + \nu_t) \frac{\partial u}{\partial y} \right) \quad (2)$$

In the inner region, the Reichart model is used, in which eddy viscosity, ν_t is given by:

$$\nu_t = \kappa \nu \left[\left(\frac{y u^*}{\nu} \right) - 9.7 \tanh \left(\frac{y u^*}{9.7 \nu} \right) \right] \quad (3)$$

In the outer region, the Clauser model assumes the eddy viscosity reaches a constant value.

$$(\nu_t)_{max} = c_1 U_e \delta(x) \quad (4)$$

Here, $\delta(x)$ is taken at $u = 0.994 U_\infty$, $\kappa = 0.41$, and $c_1 = 0.001$.

At the initial location, the velocity profile in the x direction is assumed to be the following piecewise function:

$$\frac{u}{U_\infty} = \begin{cases} \frac{y}{\delta} \left(\frac{3}{2} - \frac{1}{2} \left(\frac{y}{\delta} \right)^2 \right) & \frac{y}{\delta} \leq 1 \\ 1 & \frac{y}{\delta} > 1 \end{cases} \quad (5)$$

For this, the following scales will be used:

$$\nu_\infty = 1.5e10^{-6} \frac{m^2}{s}, \quad U_\infty = 30 \frac{m}{s}, \quad L = 1.0 \text{ m}, \quad \delta = 0.005 \text{ m}, \quad y_{max} = 50\delta, \quad 250 \text{ y-grid points}$$

Numerical Method Description

For the numerical solution, a coordinate transformation is applied, allowing the use of a stretched grid in the y-direction. This will allow for higher accuracy closer to the wall. To do this, the physical coordinate, y , is related to the numerical domain coordinate, η , by the following. Note that the stretching factor, s , is equal to 10.

$$y = y_{max} \frac{\sinh(s\eta)}{\sinh(s)} \quad (6)$$

To correctly map into the η coordinate, the governing equation for momentum must be transformed.

$$u \frac{\partial u}{\partial x} + \left(v - \frac{\partial v_t}{\partial y} \right) \left(\frac{\partial u}{\partial \eta} \frac{\partial \eta}{\partial y} \right) = (v + v_t) \left(\frac{\partial^2 u}{\partial \eta^2} \left(\frac{\partial \eta}{\partial y} \right)^2 + \frac{\partial u}{\partial \eta} \frac{\partial^2 \eta}{\partial y^2} \right) \quad (7)$$

The first and second derivatives of η with respect to y are as follows. These solutions were found using a symbolic solver in Python (Project 2 code).

$$\frac{\partial \eta}{\partial y} = \frac{\sinh(s)}{sy_{max} \cosh(\eta s)} \quad (8)$$

$$\frac{\partial^2 \eta}{\partial y^2} = -\frac{\sinh^2(s) \sinh(\eta s)}{sy_{max}^2 \cosh^3(\eta s)} \quad (9)$$

Spatial discretization of the transformed momentum equation (eq. 7) is solved using a 4th order central difference method. The first and second derivative of u are discretized with the following equations.

$$\left(\frac{\partial u}{\partial \eta} \right)_i \approx \frac{-u_{i+2} + 8u_{i+1} - 8u_{i-1} + u_{i-2}}{12\Delta\eta} \quad (10)$$

$$\left(\frac{\partial^2 u}{\partial \eta^2} \right)_i \approx \frac{-u_{i+2} + 16u_{i+1} - 30u_i + 16u_{i-1} - u_{i-2}}{12\Delta\eta^2} \quad (11)$$

Note that these equations use a total of four neighbors, therefore, for boundary-adjacent nodes, a second-order scheme must be used.

$$\left(\frac{\partial u}{\partial \eta} \right)_i \approx \frac{u_{i+1} - u_{i-1}}{2\Delta\eta} \quad (12)$$

$$\left(\frac{\partial^2 u}{\partial \eta^2} \right)_i \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta\eta^2} \quad (13)$$

The Crank-Nicolson scheme is used to step in the x -direction. This is a first-order difference between the current and $n + 1$ step, which is embedded within the C-N scheme, thus making it second-order accurate in the x direction.

$$\left(\frac{\partial u}{\partial x} \right)_i \approx \frac{u_i^{n+1} - u_i^n}{\Delta x} \quad (14)$$

The discretized solution can then be solved using Lower-Upper (LU) decomposition. The Crank-Nicolson solution shown above is implicit, meaning the solution of the future step is dependent on the current solution step.

$$Au^{n+1} = b \quad (15)$$

Here, A is the coefficient matrix, representing the $n + 1$ step, while b represents all n^{th} step values, all currently known values.

After each step, the y-velocity, v is updated by integrating the continuity equation (eq. 1). This is then used for each following step of the solution. This was solved using trapezoidal integration in Python.

$$v(y) = - \int_0^y \frac{\partial u}{\partial x} dy \quad (16)$$

Now, this solution can be solved for various values of x .

Results

The following figure shows normalized u and v velocity relative to wall distance at various downstream locations, up to $40 * L$ downstream of the initial location.

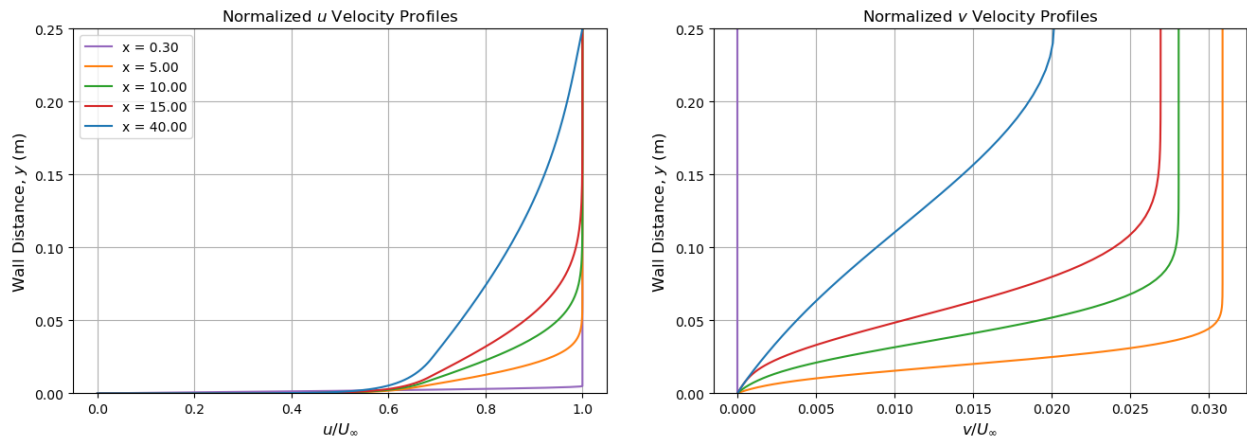


Figure 1: Normalized Turbulent Boundary Layer Velocity Profiles

Here, we see an expected boundary layer growth. However, unlike the laminar boundary layer problem from Project 2, the turbulent effects drive a much faster rate of growth.

The purple line represents the x_0 location. Recall that this is the starting point, in which the piecewise laminar assumption is being made, hence why it appears as an outlier in the velocity plots. To calculate x_0 , we must rearrange the following equation for boundary layer growth. Note $Re_x = \frac{U_\infty x}{\nu}$.

$$\delta(x) = 0.375x(Re_x)^{-1/5} \quad (17)$$

Rearranging this equation for $\delta = 0.005 \text{ m}$ gives us the following.

$$x_0 = \left(\frac{\delta}{0.375} \right)^{1.25} \left(\frac{U_\infty}{\nu} \right)^{0.25} \rightarrow x_0 = \mathbf{0.303 \text{ m}} \quad (18)$$

For model validation, we can compare the numerical solution to the theoretical boundary layer growth (eq. 17). We can also compare the numerical solution to the law of the wall.

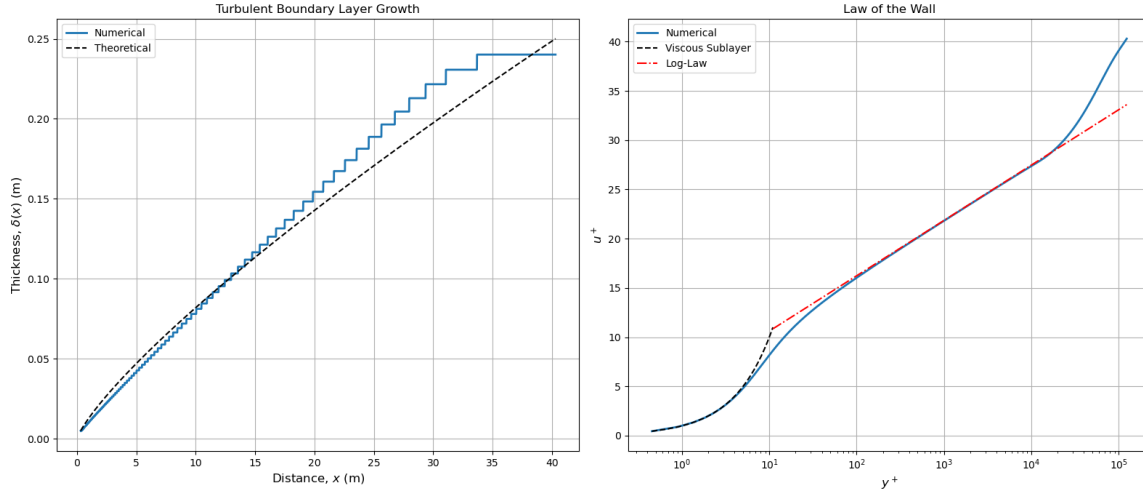


Figure 2: Comparison of Numerical Solution and Law of the Wall

These plots will be discussed further in the discussion section.

To determine the sensitivity of parameters s , κ and c_0 , I reran the solver using $\kappa \pm 0.1$, $0.2c_1$ and $5c_1$ and stored the results. Plotting the same metrics shown in Figure 3, we can see that the model is sensitive to these parameters. We can also observe the effect of changing the grid stretching. The solid black lines below show the baseline values used.

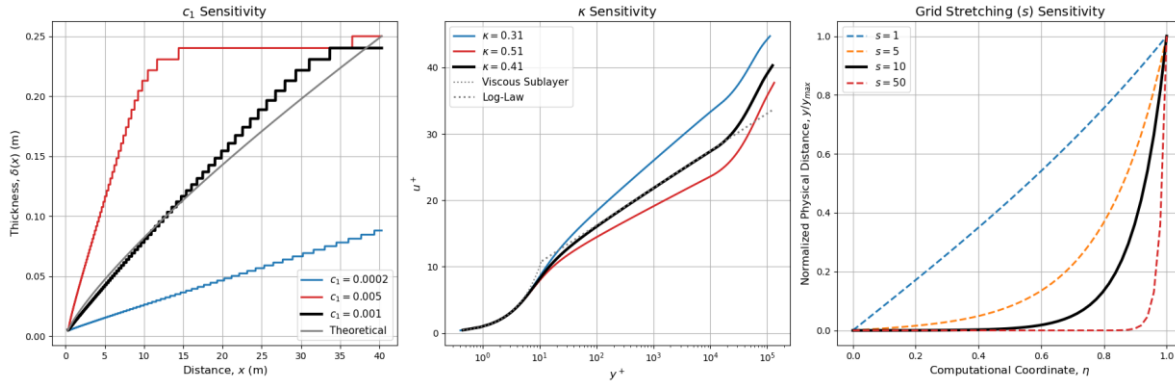


Figure 3: Sensitivity of Design Parameters

Discussion

General

As shown in Figure 1, the numerical solution experiences an expected boundary layer growth as x increases. At $x = x_0 = 0.30$, the model is still using the piecewise initial condition, hence why the boundary layer growth is much different here compared to the other lines. This shows that the turbulent effects start to dominate once the Reichardt and Clauser models take over.

To solve this problem with a 10x larger Reynolds Number, you would need to increase the number of x-direction steps. As Reynolds Number increases, the effects of turbulence become more prominent. Also, the viscous sublayer becomes thinner. With this, the stretching factor should be increased, as the viscous sublayer especially will need more steps for proper resolution.

While this solver is computationally inexpensive, it requires fine model tuning. For example, to be efficient and correct, it must have appropriate grid stretching. For the example solved in this report, the stretching factor was given, but in a real-world scenario, this would not be the case. Simply, reducing to $s = 5$ will skew the results. Also, the model requires the resolution of $0.994U_\infty$ at every step, hence why the numerical solution for boundary layer growth is a staircase curve. In reality, this curve should be smooth.

Accuracy & Stability

Similarly to Project 2, this uses a 2nd order accurate, implicit Crank-Nicolson scheme for temporal discretization, along with a 4th order central difference method for spatial discretization. This solving technique along with the use of a stretched grid gives a highly accurate solution that is also stable.

Figure 2 verifies that the boundary layer growth matches the equation provided for $\delta(x)$ (eq. 17). The increasing step size shows the effect of the stretched grid. As we move further from the wall, resolution is lost, however higher resolution is not required here.

Figure 2 also compares the numerical solution to the law of the wall. For low y^+ , $y^+ < 10$, the numerical solution does a phenomenal job of tracking the analytical solution. As expected, the numerical solution has trouble tracking at the buffer layer, in which there is a sharp kink (the analytical solution is a piecewise function). The numerical solution has a more continuous curve here, though it is still quite accurate.

Furthermore, Figure 3 shows that the accuracy of the solution is dependent on the correct design parameters. Adjusting c_1 , κ and s changes the solution drastically. As shown, c_1 affects boundary layer growth, while κ affects the accuracy of the numerical solution compared to the law of the wall. While having a higher s value is not necessarily bad, it is an example of diminishing return. As s increases, the benefit does not scale directly.

Appendix

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import lu_factor, lu_solve
import sympy as sp
from scipy.integrate import solve_bvp, trapezoid, cumulative trapezoid
```

```
#given parameters
nu_inf = 1.5e-6 #freestream kinematic viscosity (m2/s)
u_inf = 30 #freestream vel (m/s)
L = 1 # (m)
delta = 0.005 #initial boundary layer thickness (m)
Ny = 250 #updated for project 3
y_max = 50*delta #updated for project 3
s = 10 #stretch factor

#estimate x0 for turbulent boundary layer:
x0 = (delta/0.375)**1.25*(u_inf/nu_inf)**0.25

x_tot = 40*L #independent var length
dx = 0.01 #x step size
Nx = int(x_tot/dx) #x partitions

#stretched y grid
eta = np.linspace(0, 1, Ny)
d_eta = eta[1] - eta[0]
y = y_max*(np.sinh(s*eta)/np.sinh(s))

#stretched y grid derivatives
deta_dy = np.sinh(s)/(s*y_max*np.cosh(s*eta))
d2eta_dy2 = -(np.sinh(s)**2*np.sinh(s*eta))/(s*y_max**2*np.cosh(s*eta)**3)

#turbulence parameters
kappa = 0.41
ya_plus = 9.7
c1 = 0.001

print(x0)
```

```
#central diff, 4th order for interior points, 2nd order for boundary-adjacent points
du_deta = np.zeros((Ny, Ny))
d2u_deta2 = np.zeros((Ny, Ny))

for i in range(1, Ny - 1):
    if i == 1 or i == Ny - 2:
        #2nd order cd, for boundary-adjacent points
        du_deta[i, i-1] = -1/(2*d_eta)
        du_deta[i, i+1] = 1/(2*d_eta)

        d2u_deta2[i, i-1] = 1/d_eta**2
        d2u_deta2[i, i] = -2/d_eta**2
        d2u_deta2[i, i+1] = 1/d_eta**2
```

```

else:
    #4th order cd, for interior points
    du_deta[i, i-2] = 1/(12*d_eta)
    du_deta[i, i-1] = -8/(12*d_eta)
    du_deta[i, i+1] = 8/(12*d_eta)
    du_deta[i, i+2] = -1/(12*d_eta)

    d2u_deta2[i, i-2] = -1/(12*d_eta**2)
    d2u_deta2[i, i-1] = 16/(12*d_eta**2)
    d2u_deta2[i, i] = -30/(12*d_eta**2)
    d2u_deta2[i, i+1] = 16/(12*d_eta**2)
    d2u_deta2[i, i+2] = -1/(12*d_eta**2)

```

```

u = np.zeros(Ny) #init u
v = np.zeros(Ny) #init v

#at the initial location, assume that the velocity profile in the x direction is given by:
for i in range(Ny):
    if y[i]/delta <= 1:
        u[i] = u_inf*(y[i]/delta*(3/2-1/2*((y[i]/delta)**2)))
    else:
        u[i] = u_inf

x_vals = [0, 470, 970, 1470, 3970] #x steps -> x= x0 + n*dx
u_store_number = {0:u} #initialize u storing
v_store_number = {0:v} #initialize v storing

#store
x_array = np.zeros(Nx + 1)
delta_store = np.zeros(Nx + 1)
cf_store = np.zeros(Nx + 1)

x_array[0] = x0
delta_store[0] = delta

for n in range(1, Nx + 1):
    x_now = x0 + n*dx
    x_array[n] = x_now

    #friction vel
    #2nd order fwd diff at the wall
    du_deta_wall = (-3*u[0]+4*u[1]-u[2])/(2*d_eta)
    du_dy_wall = du_deta_wall*deta_dy[0]
    tau_w = nu_inf*np.abs(du_dy_wall)
    u_star = np.sqrt(tau_w)

    #boundary layer thickness calc
    idx_delta = np.argmax(u >= 0.994 * u_inf) #gemini helped with this logic
    delta_x = y[idx_delta] if idx_delta > 0 else delta

    delta_store[n] = delta_x
    cf_store[n] = tau_w/(0.5*u_inf**2)

```



```

#eddy viscosity (nu_t) and derivative
nu_t = np.zeros(Ny)
dnu_dy = np.zeros(Ny)
nut_max = c1 * u_inf * delta_x

for i in range(Ny):
    y_plus = y[i]*u_star/nu_inf
    ya_div_yp = np.clip(y_plus/ya_plus, 0, 100) #got errors due to overflow, gemini
suggested this fix..
    #reichardt
    nut_inner = kappa*nu_inf *(y_plus - ya_plus*np.tanh(ya_div_yp))
    dnu_dy_inner = kappa*u_star*(1.0 -(1.0 / np.cosh(ya_div_yp))**2)

    #clauser model for when calculated v_t from reichardt > max v_t
    if nut_inner < nut_max:
        nu_t[i] = nut_inner
        dnu_dy[i] = dnu_dy_inner
    else:
        nu_t[i] = nut_max
        dnu_dy[i] = 0.0

#diffusion convection terms + turbulence
diff_conv = np.zeros((Ny, Ny))
for i in range(1, Ny - 1):
    nu_eff = nu_inf + nu_t[i] #effective viscosity including turbulence
    v_eff = v[i] - dnu_dy[i] #effective convection velocity w/ turb visc

    j = nu_eff*deta_dy[i]**2
    k = nu_eff*d2eta_dy2[i] - v_eff*deta_dy[i]

    diff_conv[i, :] = (j*d2u_deta2[i, :] + k*du_deta[i, :]) #rearranged PDE, du/dx = ...
(see eq. 7 in report)

#Au=b
A = np.diag(u) - diff_conv*dx/2 #C-N: step n+1
b = (np.diag(u) + diff_conv*dx/2)*u #C-N: u*du/dx at step n

b[0] = 0 # u=0 at wall
b[-1] = u_inf # u=u_inf at freestream

#top boundary
A[-1, :] = 0
A[-1, -1] = 1
#bottom boundary
A[0, :] = 0
A[0, 0] = 1

u_new = lu_solve((lu_factor(A)), b) #lu solve for u

#update v using continuity eq, du/dx + dv/dy = 0
du_dx = (u_new - u)/dx
v_new = cumulative_trapezoid(-du_dx, x=y, initial=0)

```

```

u = u_new
v = v_new

u_store_numer[n]=u
v_store_numer[n]=v

```

```

#velocity plots
fig, axs = plt.subplots(1, 2, figsize=(16, 5))
colors = ['tab:purple', 'tab:orange', 'tab:green', 'tab:red', 'tab:blue']

#velocity plot
for idx, n in enumerate(x_vals):
    x_now = x_array[n]
    axs[0].plot(u_store_numer[n]/u_inf, y, '-', color=colors[idx], label=f'x = {x_now:.2f}')
axs[0].set_title('Normalized $u$ Velocity Profiles', fontsize=12)
axs[0].set_xlabel('$u / U_{\\infty}$', fontsize=12)
axs[0].set_ylabel('Wall Distance, $y$ (m)', fontsize=12)
#axs[0].set_ylim(0, 0.01)
axs[0].set_ylim(0, 0.25)
axs[0].legend()
axs[0].grid()

for idx, n in enumerate(x_vals):
    x_now = x_array[n]
    axs[1].plot(v_store_numer[n], y, '-', color=colors[idx], label=f'x = {x_now:.2f}')
axs[1].set_title('Normalized $v$ Velocity Profiles', fontsize=12)
axs[1].set_xlabel('$v / U_{\\infty}$', fontsize=12)
axs[1].set_ylabel('Wall Distance, $y$ (m)', fontsize=12)
axs[1].set_ylim(0, 0.25)
axs[1].grid()

```

```

#boundary layer thickness and law of wall check
fig, axs = plt.subplots(1, 2, figsize=(16, 7))
#boundary layer thickness
Re_x = (u_inf*x_array)/nu_inf
delta_theory = 0.375*x_array*(Re_x)**(-1/5)
axs[0].plot(x_array, delta_store, linewidth=2, label='Numerical')
axs[0].plot(x_array, delta_theory, '--', color='k', label='Theoretical')
axs[0].set_title('Turbulent Boundary Layer Growth', fontsize=12)
axs[0].set_xlabel('Distance, $x$ (m)', fontsize=12)
axs[0].set_ylabel('Thickness, $\\delta(x)$ (m)', fontsize=12)
axs[0].legend()
axs[0].grid()

#law of the wall
n_final = x_vals[-1]
u_final = u_store_numer[n_final]

#recalc u* for the final step
du_deta_wall = (-3*u_final[0] + 4*u_final[1] -u_final[2])/(2*d_eta)
du_dy_wall = du_deta_wall*deta_dy[0]
u_star_final = np.sqrt(nu_inf * np.abs(du_dy_wall))

```

```

#dimensionless quantities (exclude y=0 to avoid log(0))
y_plus_final = y[1:]*u_star_final / nu_inf
u_plus_final = u_final[1:] / u_star_final
axs[1].semilogx(y_plus_final, u_plus_final, linewidth=2, label='Numerical')

#y_plus_k31 = np.copy(y_plus_final) #for sensitivity
#u_plus_k31 = np.copy(u_plus_final)

#y_plus_k51 = np.copy(y_plus_final) #for sensitivity
#u_plus_k51 = np.copy(u_plus_final)

# Stash low c1 variation
#delta_c1_low = np.copy(delta_store) #for sensitivity

# Stash high c1 variation
#delta_c1_high = np.copy(delta_store) #for sensitivity

#theoretical Regions
y_visc = np.linspace(min(y_plus_final), 11, 50)
axs[1].semilogx(y_visc, y_visc, '--', color='k', label='Viscous Sublayer')
y_log = np.linspace(11, max(y_plus_final), 100)
u_log = (1/kappa) * np.log(y_log) + 5.0

axs[1].semilogx(y_log, u_log, '-.', color='r', label='Log-Law')
axs[1].set_title('Law of the Wall', fontsize=12)
axs[1].set_xlabel('$y^+$', fontsize=12)
axs[1].set_ylabel('$u^+$', fontsize=12)
axs[1].legend()
axs[1].grid()

plt.tight_layout()
plt.show()

```

```

#SENSITIVITY ANALYSIS PLOTS
#IMPORTANT, MUST rerun ABOVE CELLS FIRST TO POPULATE THE VARIATIONS IN THE STORED ARRAYS
fig, axs = plt.subplots(1, 3, figsize=(18, 6))

#c1 sensitivity
Re_x = (u_inf * x_array)/nu_inf
delta_theory = 0.375*x_array*(Re_x)**(-1/5)

#c1 sensitivity
axs[0].plot(x_array, delta_c1_low, '-', color='tab:blue', linewidth=2, label='$c_1 = 0.0002$')
axs[0].plot(x_array, delta_c1_high, '-', color='tab:red', linewidth=2, label='$c_1 = 0.005$')

#default
axs[0].plot(x_array, delta_store, '-', color='black', linewidth=3, label='$c_1 = 0.001$')
axs[0].plot(x_array, delta_theory, '-', color='gray', linewidth=2, label='Theoretical')

axs[0].set_title('$c_1$ Sensitivity', fontsize=14)
axs[0].set_xlabel('Distance, $x$ (m)', fontsize=12)
axs[0].set_ylabel('Thickness, $\delta(x)$ (m)', fontsize=12)
axs[0].legend(fontsize=11)

```

```

axs[0].grid(True)

#kappa sensitivity
axs[1].semilogx(y_plus_k31, u_plus_k31, '-', color='tab:blue', linewidth=2, label='$\\kappa = 0.31$')
axs[1].semilogx(y_plus_k51, u_plus_k51, '-', color='tab:red', linewidth=2, label='$\\kappa = 0.51$')

#default
axs[1].semilogx(y_plus_final, u_plus_final, '-', color='black', linewidth=3, label='$\\kappa = 0.41$')

#theory
y_visc = np.linspace(min(y_plus_final), 11, 50)
axs[1].semilogx(y_visc, y_visc, ':', color='tab:gray', label='Viscous Sublayer')

y_log = np.linspace(11, max(y_plus_final), 100)
u_log = (1/0.41) * np.log(y_log) + 5.0
axs[1].semilogx(y_log, u_log, ':', color='tab:gray', linewidth=2, label='Log-Law')

axs[1].set_title('$\\kappa$ Sensitivity', fontsize=14)
axs[1].set_xlabel('$y^+$', fontsize=12)
axs[1].set_ylabel('$u^+$', fontsize=12)
axs[1].legend(fontsize=11)
axs[1].grid(True)

#s sensitivity
eta = np.linspace(0, 1, 50)
s_test_vals = [1, 5, 10, 50]
colors_s = ['tab:blue', 'tab:orange', 'black', 'tab:red']

for idx, s_test in enumerate(s_test_vals):
    y_test = np.sinh(s_test * eta) / np.sinh(s_test)

    # Match baseline formatting (s=10)
    linewidth = 3 if s_test == 10 else 2
    linestyle = '-' if s_test == 10 else '--'
    label_str = f'$s = {s_test}$' if s_test == 10 else f'$s = {s_test}$'

    axs[2].plot(eta, y_test, linestyle=linestyle, linewidth=linewidth, color=colors_s[idx],
label=label_str)

axs[2].set_title('Grid Stretching ($s$) Sensitivity', fontsize=14)
axs[2].set_xlabel('Computational Coordinate, $\\eta$', fontsize=12)
axs[2].set_ylabel('Normalized Physical Distance, $y/y_{\\max}$', fontsize=12)
axs[2].legend(fontsize=11)
axs[2].grid(True)

plt.tight_layout()
plt.show()

```